

2.1 Structure of Relational Databases

A relational database consists of a collection of tables, each of which is assigned a unique name.

representing relationships.

In general, a row in a table represents a *relationship* among a set of values. Since a table is a collection of such relationships, there is a close correspondence between the concept of *table* and the mathematical concept of *relation*, from which the relational data model takes its name. In mathematical terminology, a *tuple* is simply a sequence (or list) of values. A relationship between n values is represented mathematically by an n -*tuple* of values, i.e., a tuple with n values, which corresponds to a row in a table.

Screen clipping taken: 03-06-2026 20:41

Thus, in the relational model the term *relation* is used to refer to a table, while the term *tuple* is used to refer to a row. Similarly, the term *attribute* refers to a column of a table.

Screen clipping taken: 03-06-2026 20:42

We use the term *relation instance* to refer to a specific instance of a relation, i.e., containing a specific set of rows. The instance of *instructor* shown in Figure 2.1 has 12 tuples, corresponding to 12 instructors.

Screen clipping taken: 03-06-2026 20:42

For each attribute of a relation, there is a set of permitted values, called the domain of that attribute. A domain is atomic if elements of the domain are considered to be indivisible units.

The null value is a special value that signifies that the value is unknown or does not exist.

Database Schema

Screen clipping taken: 03-06-2026 20:45

When we talk about a database, we must differentiate between the *database schema*, which is the logical design of the database, and the *database instance*, which is a snapshot of the data in the database at a given instant in time.

Screen clipping taken: 03-06-2026 20:46

Keys

Screen clipping taken: 03-06-2026 21:54

A **superkey** is a set of one or more attributes that, taken collectively, allow us to identify uniquely a tuple in the relation.

- A **superkey** of a relation is a set of one or more attributes whose values are guaranteed to identify tuples in the relation uniquely. A candidate key is a minimal superkey, that is, a set of attributes that forms a superkey, but none of whose subsets is a superkey. One of the candidate keys of a relation is chosen as its **primary key**.
- A **foreign key** is a set of attributes in a referencing relation, such that for each tuple in the referencing relation, the values of the foreign key attributes are guaranteed to occur as the primary key value of a tuple in the referenced relation.
- A **schema diagram** is a pictorial depiction of the schema of a database that shows the relations in the database, their attributes, and primary keys and foreign keys.
- The **relational query languages** define a set of operations that operate on tables, and output tables as their results. These operations can be combined to get expressions that express desired queries.
- The **relational algebra** provides a set of operations that take one or more relations as input and return a relation as an output. Practical query languages such as SQL are based on the relational algebra, but add a number of useful syntactic features.

Screen clipping taken: 04-06-2026 07:12

SQL (Structured Query Language).

The SQL language has several parts:

- **Data-definition language (DDL).** The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- **Data-manipulation language (DML).** The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
- **Integrity.** The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.
- **View definition.** The SQL DDL includes commands for defining views.
- **Transaction control.** SQL includes commands for specifying the beginning and ending of transactions.
- **Embedded SQL and dynamic SQL.** Embedded and dynamic SQL define how SQL statements can be embedded within general-purpose programming languages, such as C, C++, and Java.
- **Authorization.** The SQL DDL includes commands for specifying access rights to relations and views.

Screen clipping taken: 04-06-2026 07:22

SQL Data Definition

The set of relations in a database must be specified to the system by means of a data-definition language (DDL). The SQL DDL allows specification of not only a set of relations, but also information about each relation, including:

- The schema for each relation.
- The types of values associated with each attribute.
- The integrity constraints.
- The set of indices to be maintained for each relation.

- The security and authorization information for each relation.
- The physical storage structure of each relation on disk.

Screen clipping taken: 04-06-2026 09:38

3.2.1 Basic Types

The SQL standard supports a variety of built-in types, including:

- **char(*n*)**: A fixed-length character string with user-specified length *n*. The full form, **character**, can be used instead.
- **varchar(*n*)**: A variable-length character string with user-specified maximum length *n*. The full form, **character varying**, is equivalent.
- **int**: An integer (a finite subset of the integers that is machine dependent). The full form, **integer**, is equivalent.
- **smallint**: A small integer (a machine-dependent subset of the integer type).
- **numeric(*p*, *d*)**: A fixed-point number with user-specified precision. The number consists of *p* digits (plus a sign), and *d* of the *p* digits are to the right of the decimal point. Thus, **numeric(3,1)** allows 44.5 to be stored exactly, but neither 444.5 or 0.32 can be stored exactly in a field of this type.
- **real, double precision**: Floating-point and double-precision floating-point numbers with machine-dependent precision.
- **float(*n*)**: A floating-point number, with precision of at least *n* digits.

Screen clipping taken: 04-06-2026 09:41

Each type may include a special value called the null value. A null value indicates an absent value that may exist but be unknown or that may not exist at all.

- **primary key** ($A_{j_1}, A_{j_2}, \dots, A_{j_m}$): The **primary-key** specification says that attributes $A_{j_1}, A_{j_2}, \dots, A_{j_m}$ form the primary key for the relation. The primary-key attributes are required to be *nonnull* and *unique*; that is, no tuple can have a null value for a primary-key attribute, and no two tuples in the relation can be equal on all the primary-key attributes. Although the primary-key

specification is optional, it is generally a good idea to specify a primary key for each relation.

- **foreign key** ($A_{k_1}, A_{k_2}, \dots, A_{k_n}$) **references** *s*: The **foreign key** specification says that the values of attributes ($A_{k_1}, A_{k_2}, \dots, A_{k_n}$) for any tuple in the relation must correspond to values of the primary key attributes of some tuple in relation *s*.
- **not null**: The **not null** constraint on an attribute specifies that the null value is not allowed for that attribute; in other words, the constraint excludes the null value from the domain of that attribute. For example, in Figure 3.1, the **not null** constraint on the *name* attribute of the *instructor* relation ensures that the name of an instructor cannot be null.

SQL prevents any update to the database that violates an integrity constraint. For example, if a newly inserted or modified tuple in a relation has null values for any primary-key attribute, or if the tuple has the same value on the primary-key attributes as does another tuple in the relation, SQL flags an error and prevents the update. Similarly, an insertion of a *course* tuple with a *dept_name* value that does not appear in the *department* relation would violate the foreign-key constraint on *course*, and SQL prevents such an insertion from taking place.

Screen clipping taken: 04-06-2026 09:54

A newly created relation is empty initially. We can use the insert command to load data into the relation.

```
insert into instructor
values (10211, 'Smith', 'Biology', 66000);
```

Screen clipping taken: 04-06-2026 09:55

The values are specified in the order in which the corresponding attributes are listed in the relation schema.

We can use the **delete** command to delete tuples from a relation. The command

```
delete from student;
```

would delete all tuples from the student relation

```
create table department
(dept_name  varchar (20),
 building   varchar (15),
 budget     numeric (12,2),
 primary key (dept_name));

create table course
(course_id   varchar (7),
 title      varchar (50),
 dept_name  varchar (20),
 credits     numeric (2,0),
 primary key (course_id),
 foreign key (dept_name) references department);

create table instructor
(ID          varchar (5),
 name        varchar (20) not null,
 dept_name   varchar (20),
 salary      numeric (8,2),
 primary key (ID),
 foreign key (dept_name) references department);
```

```

create table section
(course_id    varchar (8),
sec_id       varchar (8),
semester     varchar (6),
year         numeric (4,0),
building     varchar (15),
room_number  varchar (7),
time_slot_id varchar (4),
primary key (course_id, sec_id, semester, year),
foreign key (course_id) references course);

create table teaches
(ID          varchar (5),
course_id    varchar (8),
sec_id       varchar (8),
semester     varchar (6),
year         numeric (4,0),
primary key (ID, course_id, sec_id, semester, year),
foreign key (course_id, sec_id, semester, year) references section,
foreign key (ID) references instructor);

```

Figure 3.1 SQL data definition for part of the university database.

To remove a relation from an SQL database, we use the **drop table** command. The **drop table** command deletes all information about the dropped relation from the database. The command

```
drop table r;
```

is a more drastic action than

```
delete from r;
```

We use the **alter table** command to add attributes to an existing relation. All tuples in the relation are assigned *null* as the value for the new attribute. The form of the **alter table** command is

```
alter table r add A D;
```

where *r* is the name of an existing relation, *A* is the name of the attribute to be added, and *D* is the type of the added attribute. We can drop attributes from a relation by the command

```
alter table r drop A;
```

where *r* is the name of an existing relation, and *A* is the name of an attribute of the relation. Many database systems do not support dropping of attributes, although they will allow an entire table to be dropped.

Screen clipping taken: 04-06-2026 10:00

Basic Structure of SQL Queries

Screen clipping taken: 04-06-2026 10:06

The basic structure of an SQL query consists of three clauses:

1. Select
2. From
3. Where

The query takes as its input the relations listed in the from clause, operates on them as specified in the where and select clauses, and then produces a relation as the result.

Queries on a Single Relation:

"Find the names of all instructors." Instructor names are found in the instructor relation, so we put that relation in the from clause. The instructor's name appears in the name attribute, so we put that in the select clause.

```
select name
from instructor;
```

Screen clipping taken: 04-06-2026 10:33

Now consider another query, "Find the department names of all instructors," which can be written as:

```
select dept_name
from instructor;
```

Screen clipping taken: 04-06-2026 10:33

In those cases where we want to force the elimination of duplicates, we insert the keyword **distinct** after **select**. We can rewrite the preceding query as:

```
select distinct dept_name
from instructor;
```

Screen clipping taken: 04-06-2026 10:35

SQL allows us to use the keyword **all** to specify explicitly that duplicates are not removed:

```
select all dept_name
from instructor;
```

Since duplicate retention is the default, we shall not use **all** in our examples. To ensure the elimination of duplicates in the results of our example queries, we shall use **distinct** whenever it is necessary.

The **select** clause may also contain arithmetic expressions involving the operators **+**, **-**, *****, and **/** operating on constants or attributes of tuples. For example, the query:

```
select ID, name, dept_name, salary * 1.1
from instructor;
```

Screen clipping taken: 04-06-2026 10:38

The **where** clause allows us to select only those rows in the result relation of the **from** clause that satisfy a specified predicate. Consider the query "Find the names of all instructors in the Computer Science department who have salary greater than \$70,000." This query can be written in SQL as:

Screen clipping taken: 04-06-2026 10:39

```
select name
from instructor
where dept_name = 'Comp. Sci.' and salary > 70000;
```

Screen clipping taken: 04-06-2026 10:39

SQL allows the use of the logical connectives **and**, **or**, and **not** in the **where** clause. The operands of the logical connectives can be expressions involving the comparison operators **<**, **<=**, **>**, **>=**, **=**, and **<>**. SQL allows us to use the comparison operators to compare strings and arithmetic expressions, as well as special types, such as date types.

Screen clipping taken: 04-06-2026 10:40

3.3.2 Queries on Multiple Relations

Screen clipping taken: 04-06-2026 10:40

In SQL, to answer the above query, we list the relations that need to be accessed in the **from** clause, and specify the matching condition in the **where** clause. The above query can be written in SQL as

```
select name, instructor.dept_name, building
from instructor, department
where instructor.dept_name= department.dept_name;
```

Screen clipping taken: 04-06-2026 10:53

We now consider the general case of SQL queries involving multiple relations. As we have seen earlier, an SQL query can contain three types of clauses, the **select** clause, the **from** clause, and the **where** clause. The role of each clause is as follows:

- The **select** clause is used to list the attributes desired in the result of a query.
- The **from** clause is a list of the relations to be accessed in the evaluation of the query.
- The **where** clause is a predicate involving attributes of the relation in the **from** clause.

A typical SQL query has the form

```
select  $A_1, A_2, \dots, A_n$ 
from  $r_1, r_2, \dots, r_m$ 
where  $P;$ 
```

Each A_i represents an attribute, and each r_i a relation. P is a predicate. If the **where** clause is omitted, the predicate P is **true**.

Screen clipping taken: 04-06-2026 10:54

Although the clauses must be written in the order **select, from, where**, the easiest way to understand the operations specified by the query is to consider the clauses in operational order: first **from**, then **where**, and then **select**.¹

Screen clipping taken: 04-06-2026 10:55

The natural join operation operates on two relations and produces a relation as the result

Consider the query “For all instructors in the university who have taught some course, find their names and the course ID of all courses they taught”, which we wrote earlier as:

```
select name, course_id
from instructor, teaches
where instructor.ID= teaches.ID;
```

This query can be written more concisely using the natural-join operation in SQL as:

```
select name, course_id
from instructor natural join teaches;
```

Both of the above queries generate the same result.

Screen clipping taken: 04-06-2026 13:48

Additional Basic Operations

The Rename Operation

The **as** clause can appear in both the **select** and **from** clauses.⁴

For example, if we want the attribute name *name* to be replaced with the name *instructor_name*, we can rewrite the preceding query as:

```
select name as instructor_name, course_id
from instructor, teaches
where instructor.ID= teaches.ID;
```

Screen clipping taken: 04-06-2026 14:19

4.2 String Operations

SQL also permits a variety of functions on character strings, such as concatenating (using “||”), extracting substrings, finding the length of strings, converting strings to uppercase (using the function **upper(s)** where *s* is a string) and lowercase (using the function **lower(s)**), removing spaces at the end of the string (using **trim(s)**) and so on. There are variations on the exact set of string functions supported by different database systems. See your database system’s manual for more details on exactly what string functions it supports.

Pattern matching can be performed on strings, using the operator **like**. We describe patterns by using two special characters:

- Percent (%): The % character matches any substring.
- Underscore (_): The _ character matches any character.

Screen clipping taken: 04-06-2026 14:20

- ‘Intro%’ matches any string beginning with “Intro”.
- ‘%Comp%’ matches any string containing “Comp” as a substring, for example, ‘Intro. to Computer Science’, and ‘Computational Biology’.
- ‘_ _ _’ matches any string of exactly three characters.
- ‘_ _ _%’ matches any string of at least three characters.

Screen clipping taken: 04-06-2026 14:21

3.4.5 Where Clause Predicates

SQL includes a **between** comparison operator to simplify **where** clauses that specify that a value be less than or equal to some value and greater than or equal to some other value. If we wish to find the names of instructors with salary amounts between \$90,000 and \$100,000, we can use the **between** comparison to write:

```
select name
from instructor
where salary between 90000 and 100000;
```

Screen clipping taken: 04-06-2026 14:23

Set Operations

The SQL operations **union**, **intersect**, and **except** operate on relations and correspond to the mathematical set-theory operations $\cup$, $\cap$, and $-$. We shall now construct queries involving the **union**, **intersect**, and **except** operations over two sets.

Screen clipping taken: 04-06-2026 14:23

Figure 3.10 The result relation, listing courses taught in Spring 2010.

3.5.1 The Union Operation

To find the set of all courses taught either in Fall 2009 or in Spring 2010, or both, we write:⁶

```
(select course_id
 from section
 where semester = 'Fall' and year= 2009)
union
(select course_id
 from section
 where semester = 'Spring' and year= 2010);
```

The **union** operation automatically eliminates duplicates, unlike the **select** clause.

Screen clipping taken: 04-06-2026 14:25

3.5.2 The Intersect Operation

To find the set of all courses taught in the Fall 2009 as well as in Spring 2010 we write:

```
(select course_id
 from section
 where semester = 'Fall' and year= 2009)
intersect
(select course_id
 from section
 where semester = 'Spring' and year= 2010);
```

The result relation, shown in Figure 3.12, contains only one tuple with CS-101. The **intersect** operation automatically eliminates duplicates. For example, if it were

Screen clipping taken: 04-06-2026 14:26

3.5.3 The Except Operation

To find all courses taught in the Fall 2009 semester but not in the Spring 2010 semester, we write:

```
(select course_id
from section
where semester = 'Fall' and year= 2009)
except
(select course_id
from section
where semester = 'Spring' and year= 2010);
```

The result of this query is shown in Figure 3.13. Note that this is exactly relation *c1* of Figure 3.9 except that the tuple for CS-101 does not appear. The **except** operation⁷ outputs all tuples from its first input that do not occur in the second input; that is, it performs set difference. The operation automatically eliminates duplicates in the inputs before performing set difference. For example, if 4 sections of ECE-101 were taught in the Fall 2009 semester and 2 sections of ECE-101 were taught in the Spring 2010 semester, the result of the **except** operation would not have any copy of ECE-101.

If we want to retain duplicates, we must write **except all** in place of **except**:

Screen clipping taken: 04-06-2026 14:26

If we want to retain duplicates, we must write **except all** in place of **except**:

```
(select course_id
from section
where semester = 'Fall' and year= 2009)
except all
(select course_id
from section
where semester = 'Spring' and year= 2010);
```

Screen clipping taken: 04-06-2026 14:27

Null Values

Null values present special problems in relational operations, including arithmetic operations, comparison operations, and set operations.

The result of an arithmetic expression (involving, for example $+$, $-$, $*$, or $/$) is null if any of the input values is null. For example, if a query has an expression $r.A + 5$, and $r.A$ is null for a particular tuple, then the expression result must also be null for that tuple.

Screen clipping taken: 04-06-2026 14:31

Since the predicate in a **where** clause can involve Boolean operations such as **and**, **or**, and **not** on the results of comparisons, the definitions of the Boolean operations are extended to deal with the value **unknown**.

- **and**: The result of *true and unknown* is *unknown*, *false and unknown* is *false*, while *unknown and unknown* is *unknown*.
- **or**: The result of *true or unknown* is *true*, *false or unknown* is *unknown*, while *unknown or unknown* is *unknown*.
- **not**: The result of *not unknown* is *unknown*.

Screen clipping taken: 04-06-2026 14:34

Aggregate Functions

Aggregate functions are functions that take a collection (a set or multiset) of values as input and return a single value. SQL offers five built-in aggregate functions:

- Average: **avg**
- Minimum: **min**
- Maximum: **max**
- Total: **sum**
- Count: **count**

The input to **sum** and **avg** must be a collection of numbers, but the other operators can operate on collections of nonnumeric data types, such as strings, as well.

Screen clipping taken: 04-06-2026 14:34

3.7.1 Basic Aggregation

Consider the query “Find the average salary of instructors in the Computer Science department.” We write this query as follows:

```
select avg (salary)
from instructor
where dept_name= 'Comp. Sci.';
```

The result of this query is a relation with a single attribute, containing a single tuple with a numerical value corresponding to the average salary of instructors in the Computer Science department. The database system may give an arbitrary name to the result relation attribute that is generated by aggregation; however, we can give a meaningful name to the attribute by using the **as** clause as follows:

```
select avg (salary) as avg_salary
from instructor
where dept_name= 'Comp. Sci.';
```

Screen clipping taken: 04-06-2026 14:39

3.7.2 Aggregation with Grouping

There are circumstances where we would like to apply the aggregate function not only to a single set of tuples, but also to a group of sets of tuples; we specify this wish in SQL using the **group by** clause. The attribute or attributes given in the **group by** clause are used to form groups. Tuples with the same value on all attributes in the **group by** clause are placed in one group.

As an illustration, consider the query “Find the average salary in each department.” We write this query as follows:

```
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name;
```

Screen clipping taken: 04-06-2026 14:40

3.7.3 The Having Clause

At times, it is useful to state a condition that applies to groups rather than to tuples. For example, we might be interested in only those departments where the average salary of the instructors is more than \$42,000. This condition does not apply to a single tuple; rather, it applies to each group constructed by the **group by** clause. To express such a query, we use the **having** clause of SQL. SQL applies predicates in the **having** clause after groups have been formed, so aggregate functions may be used. We express this query in SQL as follows:

```
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name
having avg (salary) > 42000;
```

Screen clipping taken: 04-06-2026 14:45

3.7.4 Aggregation with Null and Boolean Values

Null values, when they exist, complicate the processing of aggregate operators. For example, assume that some tuples in the *instructor* relation have a null value for *salary*. Consider the following query to total all salary amounts:

```
select sum (salary)
from instructor;
```

The values to be summed in the preceding query include null values, since some tuples have a null value for *salary*. Rather than say that the overall sum is itself *null*, the SQL standard says that the **sum** operator should ignore *null* values in its input.

In general, aggregate functions treat nulls according to the following rule: All aggregate functions except **count (*)** ignore null values in their input collection. As a result of null values being ignored, the collection of values may be empty. The **count** of an empty collection is defined to be 0, and all other aggregate operations

Screen clipping taken: 04-06-2026 14:45

return a value of null when applied on an empty collection. The effect of null values on some of the more complicated SQL constructs can be subtle.

A **Boolean** data type that can take values **true**, **false**, and **unknown**, was introduced in SQL:1999. The aggregate functions **some** and **every**, which mean exactly what you would intuitively expect, can be applied on a collection of Boolean values.

Screen clipping taken: 04-06-2026 14:46